Online Learning with Expert Advice: Hedge with Fixed Learning Rate vs. AdaHedge

Statistical Methods for Machine Learning — Assignment 1

Stefano Bruschi
Data Science for Economics, Università degli Studi di Milano

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

1 Introduction

The aim of this project is to implement from scratch and compare two algorithms for the problem of prediction with expert advice: the *Hedge* algorithm with a fixed learning rate η [3, 2] and *AdaHedge* [1], which tunes the learning rate automatically on the observed sequence of losses. The performance of Hedge crucially depends on η: a large learning rate reacts quickly and is very effective when one expert is clearly better than the others, but it is unstable on adversarial sequences; a small learning rate is safe in the worst case but slow to concentrate on the best expert. AdaHedge removes the need to choose η in advance and is designed to be optimal in the worst case while obtaining a much smaller (even constant) regret on easy instances.

Following the assignment, the two algorithms were evaluated on synthetic loss sequences generated in three regimes of different difficulty — *stochastic*, *adversarial* and *low-gap* — and Hedge was run with several values of η. For every regime the cumulative loss and the regret as a function of time are plotted, the regimes where AdaHedge improves are identified, and the observed behaviour is linked to the theory of [1, 2].

2 Problem Setting

We use the decision-theoretic online learning (DTOL) formulation of [1]. There are K experts. At each round $t = 1, \dots, T$:

1. the learner outputs a probability vector $\boldsymbol{w}_t = (w_t^1, \dots, w_t^K)$ over the experts;
2. the environment reveals the loss vector $\boldsymbol{\ell}_t = (\ell_t^1, \dots, \ell_t^K) \in [0,1]^K$;
3. the learner incurs the loss $\boldsymbol{w}_t \cdot \boldsymbol{\ell}_t = \sum_k w_t^k \ell_t^k$.

Let $L_t^k = \sum_{s \le t} \ell_s^k$ be the cumulative loss of expert k and $L_t^* = \min_k L_t^k$ the cumulative loss of the best expert after t rounds. The quantity to be minimised is the *regret*

$$R(t) = \sum_{s=1}^{t} \boldsymbol{w}_s \cdot \boldsymbol{\ell}_s - L_t^*. \tag{1}$$

In the lecture notes [2] the learner draws an expert $I_t \sim \boldsymbol{w}_t$ and the expected regret is considered. Since $\mathbb{E}[\ell_t(I_t)] = \sum_k w_t^k \ell_t^k$, the two formulations coincide in expectation; we use the expected loss $\boldsymbol{w}_t \cdot \boldsymbol{\ell}_t$ directly, which removes the extra variance due to the random draw of I_t and makes the comparison between algorithms sharper. Note that L_t^* is computed at every t separately, so the best expert in (1) may change over time.

3 Algorithms

3.1 Hedge with fixed learning rate

Hedge starts from the uniform distribution and assigns to each expert a weight that decreases exponentially with its cumulative loss:

$$w_{t+1}^k = \frac{e^{-\eta L_t^k}}{\sum_{j=1}^K e^{-\eta L_t^j}}, \qquad w_1^k = \frac{1}{K}. \tag{2}$$

This is equivalent to the multiplicative update $w_{t+1}(i) = w_t(i) e^{-\eta \ell_t(i)}$ followed by normalisation, as in the algorithm box of [2].

Mixability gap and worst-case tuning. Following [1], define the per-round *mixability gap*

$$\delta_t(\eta) = \boldsymbol{w}_t \cdot \boldsymbol{\ell}_t + \frac{1}{\eta} \ln\bigl(\boldsymbol{w}_t \cdot e^{-\eta \boldsymbol{\ell}_t}\bigr), \qquad \Delta_T(\eta) = \sum_{t=1}^T \delta_t(\eta), \tag{3}$$

and $B_T = \sum_k \frac{1}{K} e^{-\eta L_T^k}$. The second term of δ_t telescopes, so the cumulative loss of Hedge is exactly $\Delta_T(\eta) - \frac{1}{\eta} \ln B_T$. Since $B_T \geq \frac{1}{K} e^{-\eta L_T^*}$,

$$R_{\mathrm{Hedge}(\eta)}(T) \;\leq\; \frac{\ln K}{\eta} + \Delta_T(\eta) \;\leq\; \frac{\ln K}{\eta} + \frac{\eta T}{8}, \tag{4}$$

where the last step uses $0 \leq \delta_t(\eta) \leq \eta/8$ (Hoeffding's lemma [4], Lemma 1 of [1]). Minimising the right-hand side gives the worst-case tuning

$$\eta_T = \sqrt{8 \ln K / T}, \qquad R_{\mathrm{Hedge}(\eta_T)}(T) \leq \sqrt{T \ln(K)/2}, \tag{5}$$

which is optimal for large T and K.

Bound (4) makes the role of η explicit: $\ln K/\eta$ is the price of starting from a uniform prior over the experts and decreases with η, while $\Delta_T(\eta)$ is the price of mixing the experts and, in the worst case, grows with η. Choosing η_T balances the two terms for the worst sequence; the problem is that η_T requires knowing T and, on easy sequences, is much smaller than necessary.

3.2 AdaHedge

AdaHedge [1] measures the difficulty of the sequence through the mixability gap itself. Lemma 6 of the paper refines the bound on δ_t to

$$\delta_t(\eta) \leq (e-2)\,\eta\,\bigl(1 - w_t^*(\eta)\bigr), \qquad w_t^* = \max_k w_t^k, \tag{6}$$

so $\Delta_T(\eta)$ grows slowly when the weights concentrate on one expert (easy instances) and linearly when they do not (hard instances). AdaHedge (Algorithm 1) runs Hedge in *segments* with learning rates $1, \phi^{-1}, \phi^{-2}, \ldots$: inside a segment it monitors the cumulative mixability gap Δ and, as soon as Δ exceeds the budget $b(\eta) = (1/\eta + 1/(e-1)) \ln K$, it restarts Hedge from uniform weights with $\eta \leftarrow \eta/\phi$.

The guarantees of AdaHedge used in the rest of the report are taken from [1]; the number of each result in the paper is given in brackets.

Algorithm 1 AdaHedge(ϕ), $\phi > 1$ [1]

$\eta \leftarrow \phi$
for $t = 1, 2, \ldots$ **do**
  if $t = 1$ **or** $\Delta \geq b$ **then** ▷ start a new segment
    $\eta \leftarrow \eta/\phi$; $b \leftarrow \left(\frac{1}{e-1} + \frac{1}{\eta}\right) \ln K$; $\Delta \leftarrow 0$; $\boldsymbol{w} \leftarrow \left(\frac{1}{K}, \ldots, \frac{1}{K}\right)$
  end if
  output $\boldsymbol{w}$ and observe $\boldsymbol{\ell}_t$
  $\Delta \leftarrow \Delta + \boldsymbol{w} \cdot \boldsymbol{\ell}_t + \frac{1}{\eta} \ln(\boldsymbol{w} \cdot e^{-\eta \boldsymbol{\ell}_t})$
  $w^k \leftarrow w^k e^{-\eta \ell_t^k} / (\boldsymbol{w} \cdot e^{-\eta \boldsymbol{\ell}_t})$ for $k = 1, \ldots, K$
end for

Fact 1 (Regret and number of segments; Lemma 4 of [1])**.** *If, after T rounds, AdaHedge has started m segments, then*

$$R(T) < 2 \ln K \, \frac{\phi^m - 1}{\phi - 1} + m\Big(\frac{\ln K}{e-1} + \frac{1}{8}\Big).$$

Few segments mean small regret. The next results cover the worst case (Fact 2) and easy instances (Fact 3).

Fact 2 (Worst case; Theorem 5 of [1])**.** *For every sequence of losses,*

$$R(T) \leq \frac{\phi\sqrt{\phi^2 - 1}}{\phi - 1} \sqrt{\frac{4}{e-1} L_T^* \ln K} + O\big(\ln(L_T^* + 2) \ln K\big),$$

i.e. the optimal order $O(\sqrt{L_T^* \ln K})$, without knowing T or L_T^*.

Moreover, whenever AdaHedge depletes the budget of a segment, the learning rate of that segment satisfies $\eta \geq \sqrt{(e-1) \ln K / L_T^*}$ (eq. (4) of [1]).

Fact 3 (Easy instances; Theorem 9 of [1])**.** *Let $\alpha > 0$, $\delta \in (0, 1]$ and let k^* be a fixed expert. If the loss vectors are independent and $\min_{k \neq k^*} \mathbb{E}[\ell_t^k - \ell_t^{k^*}] \geq 2\alpha$ for all t, then with probability at least $1 - \delta$ AdaHedge starts at most*

$$m^* = 1 + \left\lceil \log_\phi \left(\frac{(K-1)(e-2)}{\alpha \ln K} + \frac{\ln(2K/(\alpha^2 \delta))}{4\alpha^2 \ln K} + \frac{1}{8 \ln K} \right) \right\rceil$$

segments; hence, by Fact 1, its regret is bounded by a constant *independent of T.*

Following the experiments of [1] we use $\phi = 2$ (the golden ratio minimises the constant of Fact 2, but $\phi = 2$ gives an almost identical factor, 3.46 instead of 3.33). With $\phi = 2$, AdaHedge uses the learning rates $1, \frac{1}{2}, \frac{1}{4}, \ldots$

4 Datasets: Synthetic Loss Sequences

The assignment requires synthetic losses, which allow the difficulty of the sequence to be controlled exactly. All losses are binary, $\ell_t^k \in \{0, 1\}$, which is the extreme case of $[0, 1]$ losses and maximises the mixability gap (the proof of Lemma 6 shows that δ_t is maximised by $\{0, 1\}$-valued losses). We use $K = 10$ experts and $T = 10\,000$ rounds in all regimes. With $K = 10$ there are many competitors for the best expert (so $\ln K \approx 2.30$ is not negligible), while $T = 10^4$ is long enough to separate constant regret from regret growing like $\sqrt{T}$ and keeps the running time small.

- **Stochastic** (easy). Losses are independent; expert 1 has $\ell_t^1 \sim \text{Bernoulli}(0.3)$ and the other nine experts Bernoulli(0.5). The gap of 0.2 in expected loss is large, so the cumulative losses diverge linearly and the best expert is identified quickly. This is exactly the setting of Fact 3 with $\alpha = 0.1$.
- **Adversarial** (hard). The best expert changes over time. At each round the adversary gives loss 1 to the current leader, i.e. the expert with the smallest cumulative loss L_{t-1}^k (ties broken uniformly at random), while all other experts receive independent Bernoulli(0.5) losses. Whichever expert is currently the best is therefore immediately punished and the leadership keeps changing. This is the randomised, K-expert version of the classical counter-example used in [2] and in Section 1 of [1] to show that following the leader has linear regret. The sequence depends only on the past losses of the experts and not on the learner, so it is the same for all algorithms within a repetition and the comparison is paired.
- **Low-gap** (nearly worst case). Same as the stochastic regime, but the best expert has Bernoulli(0.49) losses against Bernoulli(0.5) for the others. After T rounds the expected advantage of the best expert is $0.01\,T = 100$, of the same order as the random fluctuations of the cumulative losses ($\sqrt{T/4} = 50$ per expert). The instance is formally "easy" (Fact 3 applies with $\alpha = 0.005$), but in practice it is indistinguishable from the hard case at this horizon.

Evaluation protocol. In online learning there is no separate training and test set: the learner is evaluated on the same sequence it learns from, and the performance measure is the regret (1). Data leakage is avoided by construction because at round t the weights $\boldsymbol{w}_t$ are computed only from $\boldsymbol{\ell}_1, \ldots, \boldsymbol{\ell}_{t-1}$, before $\boldsymbol{\ell}_t$ is revealed (the code enforces this order: `weights()` is called before `update()`). To account for the randomness of the sequences, each regime is repeated 100 times with independent sequences. Every sequence is generated from the seed $(2026, \text{regime}, \text{repetition})$, so all results are exactly reproducible. We report the mean regret over the repetitions with 95% confidence intervals (± 1.96 standard errors) in the plots, and the mean $\pm$ the standard deviation across repetitions in the table.

Learning rates for Hedge. Hedge is run with $\eta = c\,\eta_T$ for $c \in \{1/4, 1, 4, 16, 64\}$, i.e. $\eta \in \{0.011, 0.043, 0.172, 0.687, 2.747\}$, where $\eta_T = \sqrt{8 \ln K / T} = 0.0429$ is the worst-case tuning (5). The grid is geometric, centred on the theoretically motivated value, and spans from a learning rate smaller than η_T to one so large that Hedge behaves almost like Follow-the-Leader. Note that the tuning η_T uses knowledge of the horizon T, which AdaHedge does not need.

5 Implementation

The code is written in Python; `numpy` is used only for vector arithmetic and random number generation and `matplotlib` only for the plots. Both algorithms are implemented from scratch with the same online interface: `weights()` returns $\boldsymbol{w}_t$ and `update(loss)` receives $\boldsymbol{\ell}_t$. The classes `Hedge` and `AdaHedge` (Algorithm 1) are in `src/algorithms.py`, the loss generators in `src/losses.py`, the online protocol, the regret and the theoretical bounds in `src/experiments.py`; `main.py` runs all the experiments and writes figures and tables to `results/`.

Numerical stability. For large η and large cumulative losses, $e^{-\eta L_t^k}$ underflows. The weights are therefore computed as a softmax of $-\eta L_t^k$ after subtracting the maximum of the exponents, and the term $\ln(\boldsymbol{w} \cdot e^{-\eta \boldsymbol{\ell}_t})$ of AdaHedge is computed with a log-sum-exp in the log domain. Rounding errors can make δ_t very slightly negative when $\boldsymbol{w}$ is concentrated on one expert; since $\delta_t \geq 0$ by Jensen's inequality, these values are clipped to 0.

Verification. Before running the experiments the implementation was checked against the theory: (i) the losses of `Hedge` coincide with the closed form (2); (ii) during its first segment AdaHedge produces exactly the same weights as Hedge with $\eta = 1$; (iii) every increment of the mixability gap satisfies $0 \le \delta_t \le \eta/8$ (Lemma 1); (iv) in all 300 runs the final regret of AdaHedge is below the bound of Fact 1 computed with the observed number of segments. The whole experiment (3 regimes $\times$ 100 repetitions $\times$ 6 algorithms) runs in about three minutes on a laptop.

6 Results

Table 1 reports the regret after $T = 10\,000$ rounds. Figures 1–3 show, for every regime, the cumulative loss (left) and the regret as a function of time (right). Figure 4 compares the final regret of Hedge as a function of η with AdaHedge and with the worst-case bound (4). Table 2 reports the behaviour of AdaHedge.

Table 1: Final regret $R(T)$, $T = 10\,000$, $K = 10$: mean $\pm$ standard deviation over 100 repetitions. The best value in each regime is in bold. Reference: worst-case bound of Hedge with η_T, $\sqrt{T\ln(K)/2} = 107.3$.

Algorithm	η	Stochastic	Adversarial	Low-gap
Hedge, $\eta = 1/4\,\eta_T$	0.011	216.5 ± 0.1	**14.4 ± 0.6**	96.2 ± 25.3
Hedge, $\eta = 1\,\eta_T$	0.043	55.6 ± 0.2	50.5 ± 0.6	79.2 ± 5.7
Hedge, $\eta = 4\,\eta_T$	0.172	15.5 ± 0.4	194.1 ± 0.6	68.9 ± 22.6
Hedge, $\eta = 16\,\eta_T$	0.687	5.9 ± 1.1	735.3 ± 1.5	**68.7 ± 30.6**
Hedge, $\eta = 64\,\eta_T$	2.747	**4.1 ± 1.8**	2079.5 ± 8.7	68.7 ± 32.1
AdaHedge ($\phi = 2$)	adaptive	6.0 ± 3.1	85.1 ± 1.7	84.4 ± 23.9

Table 2: Behaviour of AdaHedge ($\phi = 2$) over the 100 repetitions.

	Stochastic	Adversarial	Low-gap
Segments started (mean [min, max])	1.18 [1, 2]	6.00 [6, 6]	5.22 [3, 6]
Final learning rate (mean)	0.910	0.031	0.058
Runs in which expert 1 is the best in hindsight	100%	—	69%
Fact 1 bound satisfied	100%	100%	100%

Stochastic regime. The regret of every algorithm becomes constant after a few hundred rounds (Figure 1), but the level of the plateau depends strongly on η: it decreases monotonically from 216.5 for $\eta = \eta_T/4$ to 4.1 for $\eta = 64\,\eta_T$. The worst-case tuning η_T gives 55.6, about ten times more than the large learning rates. AdaHedge obtains 6.0, practically the same as Hedge with $16\,\eta_T$ (5.9) and close to the best value of the grid (4.1 for $64\,\eta_T$), without knowing that the sequence is easy: in 82% of the runs it never leaves its first segment ($\eta = 1$) and it never starts more than two. In the cumulative-loss panel, AdaHedge is indistinguishable from the best expert, while Hedge with small η stays visibly above it.

Adversarial regime. Here the ranking is reversed (Figure 2): the regret increases with η, from 14.4 for $\eta_T/4$ to 2079.5 for $64\,\eta_T$, and for the large learning rates it grows almost linearly in t, because Hedge concentrates on the current leader exactly when the adversary punishes it. AdaHedge obtains 85.1, which is larger than Hedge with η_T (50.5) but well below the worst-case guarantee 107.3 of η_T and far below the large learning rates. Its regret curve shows the effect

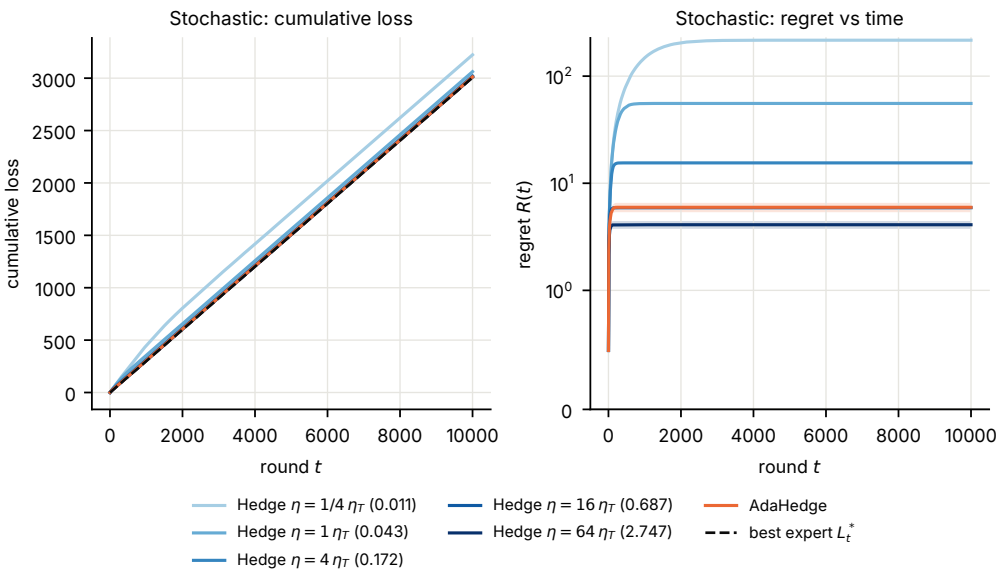


Figure 1: Stochastic regime (best expert Bernoulli(0.3), others Bernoulli(0.5)). Right panel on a symmetric-log scale; shaded bands are 95% confidence intervals of the mean.

of the restarts: it grows quickly at the beginning, when $\eta = 1$, and then bends, growing at the same rate as Hedge with η_T once the learning rate has been reduced. It always starts exactly 6 segments and ends with $\eta = 1/32 = 0.031$, a value of the same order as $\eta_T = 0.043$.

Low-gap regime. All the regret curves grow like $\sqrt{t}$ and none reaches a plateau (Figure 3). The differences between the learning rates are small compared with the variability across repetitions: all $\eta \geq 4\,\eta_T$ give ≈ 68.8, η_T gives 79.2 and $\eta_T/4$ gives 96.2. AdaHedge obtains 84.4, close to Hedge with η_T: it starts on average 5.2 segments and ends with $\eta \approx 0.058$, again of the order of η_T.

Comparison across regimes. Figure 4 summarises the three regimes. The best fixed learning rate is the largest one in the stochastic and low-gap regimes and the smallest one in the adversarial regime: no single value of η is good everywhere: the learning rate that is best in the stochastic regime ($64\,\eta_T$) has, in the adversarial regime, a regret about 25 times larger than AdaHedge and 144 times larger than the best fixed value. AdaHedge is the only algorithm that is never far from the best fixed learning rate of each regime: its final regret is within a factor 1.5 of the best value in the stochastic regime, 1.2 in the low-gap regime and 6 in the adversarial one, while remaining below the worst-case guarantee $\sqrt{T \ln(K)/2}$ in all three. Hedge with η_T, the only fixed value that is safe a priori, is ten times worse than AdaHedge in the stochastic regime.

7 Discussion

Where AdaHedge improves. As shown in Section 6, AdaHedge clearly improves over Hedge with the worst-case tuning η_T, and over every learning rate that is too small for the sequence, in the *stochastic* regime, where one expert is significantly better than the others. It does *not* improve over η_T in the adversarial and low-gap regimes, where it behaves like a worst-case tuned algorithm; its advantage there is robustness, since it avoids the catastrophic regret of the large learning rates that are optimal in the easy regime.

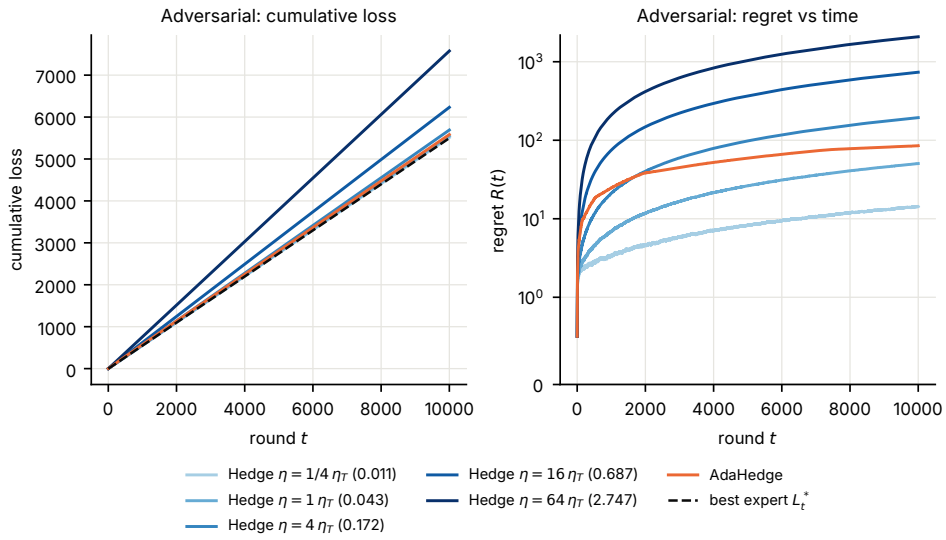


Figure 2: Adversarial regime (the current leader always receives loss 1). Right panel on a symmetric-log scale.

Link with the theory.

- *Hedge on easy sequences.* When the weights concentrate on the best expert, $B_T \approx \frac{1}{K} e^{-\eta L_T^*}$, so the first inequality in (4) is almost tight: $R(T) \approx \ln K/\eta + \Delta_T(\eta)$. By (6), $\Delta_T(\eta)$ stays bounded once $w_t^* \to 1$, and the regret is dominated by $\ln K/\eta$. This is exactly what we observe in the stochastic regime: $\ln K/\eta = 214.6$ for $\eta_T/4$ (observed 216.5) and 53.6 for η_T (observed 55.6). The regret is constant in T for every η, but the constant is inversely proportional to η, so the worst-case tuning is far too conservative, as noted in Section 5.3 of [1].

- *Hedge on hard sequences.* In the adversarial regime the weights never concentrate, so δ_t remains close to its maximum and $\Delta_T(\eta)$ grows like ηT: the observed regrets for the three largest learning rates (194, 735, 2079) follow the increasing branch of the bound $\ln K/\eta + \eta T/8$ in Figure 4. For small η the observed regret is far below the bound because the adversary keeps the cumulative losses of all experts close to each other, so mixing them costs almost nothing.

- *AdaHedge on easy sequences.* The stochastic regime satisfies the assumptions of Fact 3 with $\alpha = 0.1$, which (for $\delta = 0.05$) guarantees at most $m^* = 9$ segments and therefore constant regret. The observed number of segments (1–2) is much smaller than this conservative guarantee, and the observed regret (6.0) agrees with Fact 1, whose bound is 6.1 for $m = 1$ and 16.7 for $m = 2$. The mechanism is the one of Lemma 6: with $\eta = 1$ the weights concentrate on the best expert after a few rounds, the mixability gap stops growing and the budget is never depleted, so AdaHedge keeps the large learning rate that suits the sequence.

- *AdaHedge on hard sequences.* In the adversarial regime the budget is depleted again and again, which forces η down. By eq. (4) of [1] (recalled after Fact 2), when a budget is depleted $\eta \geq \sqrt{(e-1)\ln K/L_T^*} = 0.027$ (with $L_T^* \approx 5497$); the final value $1/32 = 0.031$ is consistent with this, i.e. AdaHedge automatically reaches the scale $\sqrt{\ln K/L_T^*}$ of the worst-case tuning. Its regret 85.1 is far below the bound of Fact 2 (leading term ≈ 595) and of Fact 1 with $m = 6$ (298.9). The extra regret with respect to Hedge with η_T is the price of adaptivity: the early segments with $\eta = 1, \frac{1}{2}, \ldots$ are too aggressive for this sequence, and each restart forgets the weights learnt so far.

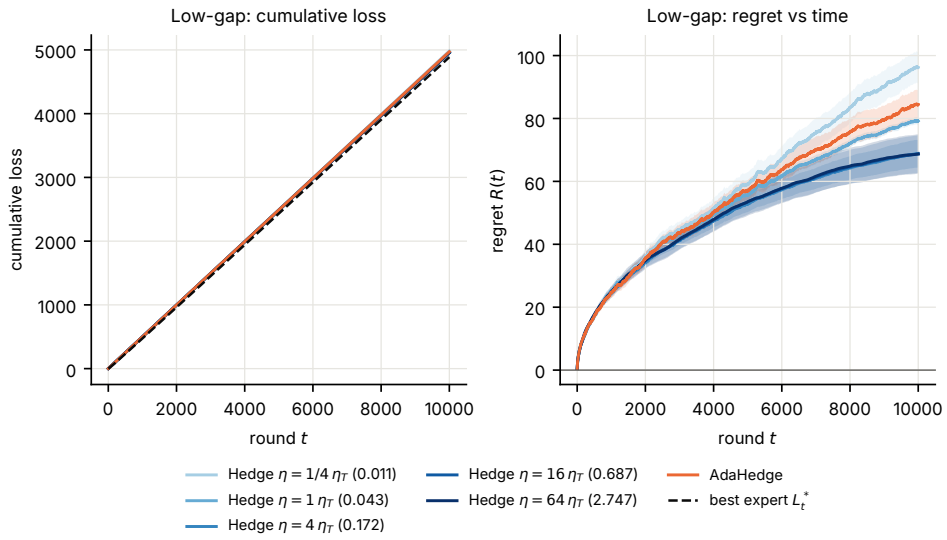


Figure 3: Low-gap regime (best expert Bernoulli(0.49), others Bernoulli(0.5)). Right panel on a linear scale.

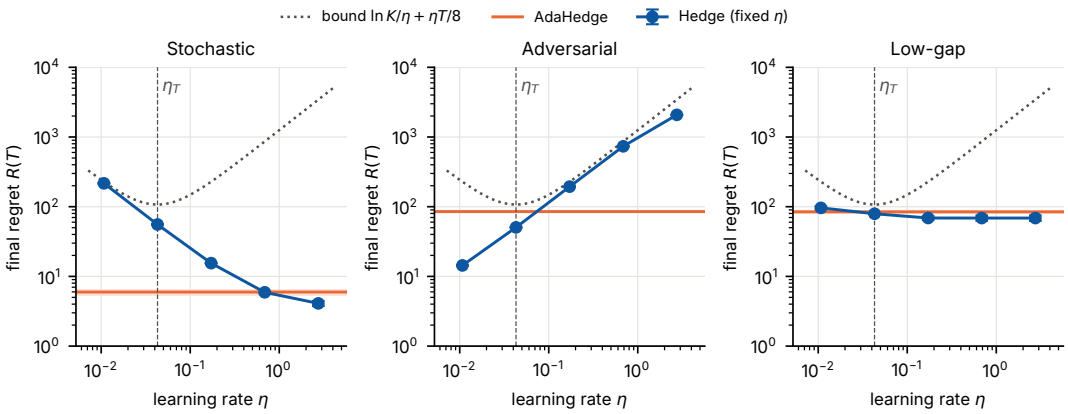


Figure 4: Final regret of Hedge as a function of the fixed learning rate (blue, with 95% confidence intervals), compared with AdaHedge (orange line and band) and with the worst-case bound $\ln K/\eta + \eta T/8$ (dotted). The dashed vertical line marks η_T.

- *Low-gap.* Fact 3 still applies formally with $\alpha = 0.005$, but it gives $m^* = 18$ segments and, through Fact 1, a constant of order 10^6, which is meaningless for $T = 10^4$. Accordingly, AdaHedge behaves as in the worst case: the mixability gap keeps growing, about five segments are started and the final learning rate is of the order of η_T. With such a small gap the best expert cannot be distinguished from the others in 10^4 rounds (it is the best one in hindsight only in 69% of the runs) and the regret grows like $\sqrt{T}$ for all algorithms, in agreement with the worst-case order $O(\sqrt{T \ln K})$. The good result of the large learning rates here is not in contradiction with the theory: the losses are independent and identically distributed, so the leader does not change adversarially and following it is not penalised; the regret is caused only by the random fluctuations of the cumulative losses.

Positive and negative aspects. The main positive aspect of AdaHedge is that it combines the two regimes without any prior information: it obtains constant regret on the easy stochastic sequence, like the most aggressive learning rates, and keeps the worst-case guarantee $O(\sqrt{L_T^* \ln K})$ on the adversarial one, without knowing T (which η_T requires) or L_T^*. The

negative aspects are: (i) on hard sequences its regret is larger than that of Hedge tuned with knowledge of T, reflecting the larger constant in Fact 2 and the cost of starting with $\eta = 1$; (ii) the restarts discard all the information collected in the previous segment, which is visible as a steep initial part of the regret curve in the adversarial regime; (iii) its regret has a larger variance across repetitions than that of Hedge with a fixed η (Table 1), because the number of segments and the times at which they start depend on the sequence.

Limitations of the study. The conclusions are based on synthetic sequences with binary losses, $K = 10$ experts and a single horizon $T = 10^4$; the adversarial sequence is oblivious to the learner. The regret reported is the expected regret of the randomised learner; the realised loss of a learner that samples $I_t \sim \boldsymbol{w}_t$ would have additional variance but the same mean.

8 Conclusion

Hedge with a fixed learning rate and AdaHedge were implemented from scratch and compared on stochastic, adversarial and low-gap loss sequences. The experiments confirm the main message of [1]: the best fixed learning rate depends on the difficulty of the sequence (large on easy sequences, small on adversarial ones), and the worst-case tuning η_T is overly conservative when one expert is clearly better than the others. AdaHedge adapts to the regime automatically: it achieves constant regret in the stochastic regime, about ten times smaller than Hedge with η_T, and it behaves like a worst-case tuned algorithm in the adversarial and low-gap regimes, staying within the $O(\sqrt{T \ln K})$ guarantee at the cost of a moderate constant factor with respect to the best fixed learning rate. The observed numbers of segments, final learning rates and regret levels are consistent with Lemmas 1 and 6 of [1] and with Facts 1–3.

References

[1] T. van Erven, P. D. Grünwald, W. M. Koolen, S. de Rooij. *Adaptive Hedge.* Advances in Neural Information Processing Systems (NeurIPS), 2011. arXiv:1110.6416.

[2] T. R. Cesari, N. Cesa-Bianchi. *Hedge and Exp3 for sequential decision-making.* Lecture notes, Università degli Studi di Milano, 2025.

[3] Y. Freund, R. E. Schapire. *A decision-theoretic generalization of on-line learning and an application to boosting.* Journal of Computer and System Sciences, 55:119–139, 1997.

[4] N. Cesa-Bianchi, G. Lugosi. *Prediction, Learning, and Games.* Cambridge University Press, 2006.